CE 216 – Fundamental Topics in Programming

Associate Prof. Dr. Kaya OĞUZ

PlayForge Manager

Design and Architecture Document

Milestone #1

Team Members: Mirza İşi, Sepehr Rahimi, Doruk Faydalı, Çağan Parlapan

Submission Date: 15.03.2026

Spring 2026

Contents

1 Milestone Scope and Requirement Summary **3**

2 Functional View of the System and Key Requirements **3**

2.1 Functional View of the System 3

2.2 Key Functional Requirements 4

2.3 Key Non-Functional Requirements 4

3 Architectural Drivers **5**

4 Proposed Layered Architecture **5**

4.1 Layer Definitions 5

4.2 Dependency Rule 6

5 Core Abstractions and Interfaces **6**

5.1 Classification Overview 7

5.2 Responsibility Summary 7

5.3 Representative Responsibility Matrix 8

5.4 Ownership Boundaries 10

5.5 Key Design Notes 10

6 Football Concrete Design **11**

6.1 Main Football Components 11

6.2 Football Entities and Composition 12

6.3 Football Initialization Notes 12

6.4 Mapping to the Core Model 13

7 Future Sport Extension Strategy **13**

8 UI Architecture and Screen Map **15**

9 Main Workflows **16**

9.1 New Game Initialization 16
9.2 Weekly Progression 17
9.3 Match Preparation, Simulation, and Result Update 18
9.4 Sequence View of New Game Initialization 18

10 Class Relationships **19**
10.1 Shared Framework Relationships 20
10.2 Football Realization of the Framework 21

11 OO Principles and Design Decisions **21**

12 Risks, Assumptions, and Open Questions **23**
12.1 Risks 23
12.2 Assumptions 23

13 Rubric and Requirement Traceability **23**

14 Design Process and Ownership **24**
14.1 Initial Design Baseline 24
14.2 Section and Diagram Ownership 25

Conclusion **25**

Declaration on the Use of AI Assistance **25**

1 Milestone Scope and Requirement Summary

This document is about the design and architecture of **PlayForge Manager**, a GUI-based sports manager game. Football is the first sport that will be implemented, but the system is designed so that another sport can be added later without changing the overall structure of the application.

The purpose of this milestone is to define the software design. This document covers the main parts of the system, the layered architecture, the core abstractions and interfaces, the football-specific design, the user interface structure, the main workflows, and the relationships between the main classes.

The main goal of this project as defined in the project instructions is extensibility. The application should not be tied only to football. Instead, it should offer a shared structure that can support different sports while keeping the user interface and the main game flow consistent.

The planned implementation language is **Java**. The final application is intended to run as a desktop program installed by a setup executable on a Windows machine with a recent Java Runtime installed. In the current design, **JavaFX** is the GUI framework which is planned to use, although the architecture keeps the UI layer separate enough that the underlying parts could still be changed later when needed.

2 Functional View of the System and Key Requirements

2.1 Functional View of the System

The game starts when the user creates a new session and chooses a sport. In the first version of the project, the selected sport is Football. After the choice is made, the system creates the sport context, generates a league, creates teams, players, and coaches, and then produces a fixture list for the season.

During the season, the user manages one team. The user can inspect the squad, check fixtures, view the league table, choose a lineup, and set a tactic before matches. Each week, training is applied, player availability is updated, the next match is prepared, and the match is simulated according to the rules of the selected sport. After the match, the result is stored, the standings are updated, and player conditions such as injuries are processed by the program. The season continues until the competition is completed and

at the end, the user sees the final outcome and may continue into another season, or may not.

2.2 Key Functional Requirements

- The system shall allow the user to start a new game and choose a sport.
- The system shall generate a league, teams, players, and coaches for the selected sport.
- The system shall create and maintain a season fixture list.
- The system shall allow the user to manage one team during the season.
- The system shall allow the user to view squad information, fixtures, and league standings.
- The system shall allow the user to select lineups and tactics before matches.
- The system shall simulate matches according to the rules of the selected sport.
- The system shall update results, standings, and player status after each match.
- The system shall support continuation into future seasons.

2.3 Key Non-Functional Requirements

- The application shall provide a clean and modern graphical user interface as much as JavaFX allows.
- The design shall support multiple sports through abstraction and extensibility.
- The user interface shall remain independent from sport-specific concrete classes.
- The architecture shall separate presentation logic, workflow coordination, shared domain logic, and sport-specific behavior.
- The system shall support generated male and female names for players and coaches, team names, and logos through a dedicated support abstraction.
- The final implementation shall target a Java-based desktop environment.
- The design shall remain maintainable and testable for later milestones.

3 Architectural Drivers

The design of PlayForge Manager is shaped by four main drivers and those are extensibility, separation of concerns, maintainability, and testability.

Extensibility is the most important one. Football is only the first concrete sport. The software therefore needs shared abstractions rather than football-only classes. This allows another sport, such as Volleyball, to be added later without redesigning the whole system from scratch.

Separation of concerns also matters a lot. The UI, the game flow, and the sport-specific rules must not be mixed together. The UI should deal with presentation and user input. The application layer should coordinate workflows. The domain layer should define shared concepts. The sport module should implement the rules of a specific sport.

Maintainability matters because the project continues through later milestones. Clear class responsibilities, stable interfaces, and controlled dependencies make later work easier.

Testability is important because the next milestone requires foundation classes and unit tests. A design built around interfaces and isolated policies makes it easier to test components such as match engines, schedulers, standings policies, and injury policies.

4 Proposed Layered Architecture

The system uses a layered structure so that user interaction, workflow coordination, shared game concepts, sport-specific behavior, and support services stay separate.

4.1 Layer Definitions

Presentation and UI Layer. This layer contains screens and user interaction logic. It displays data and collects input, but it does not contain football-specific logic. In the planned implementation, this layer will be built with JavaFX views, controllers, and small view-facing models.

Application Layer. This layer coordinates use cases such as starting a new game, progressing through the week, preparing matches, and updating the game state. The main service-level operations in the current design are game initialization, sport selection, squad viewing, lineup selection, fixture retrieval, standings retrieval, match processing, and post-match summary generation.

Domain / Core Layer. This layer defines the stable abstractions shared by all sports,

such as `Sport`, `League`, `Season`, `Match`, `Team`, `Scheduler`, and `MatchEngine`.

Sport Modules. This layer contains the concrete implementation of a selected sport. In this milestone, Football is the first concrete module.

Infrastructure / Assets Layer. This layer supports the rest of the system with data and technical services such as asset loading, name generation, logo support, and future persistence.

4.2 Dependency Rule

The main rule is very simple, the UI depends only on application services and view-facing data. It must never depend directly on concrete football classes. The application layer coordinates workflows, the domain layer defines stable contracts, and the sport module implements those contracts. Infrastructure supports the other layers but does not control the main flow.

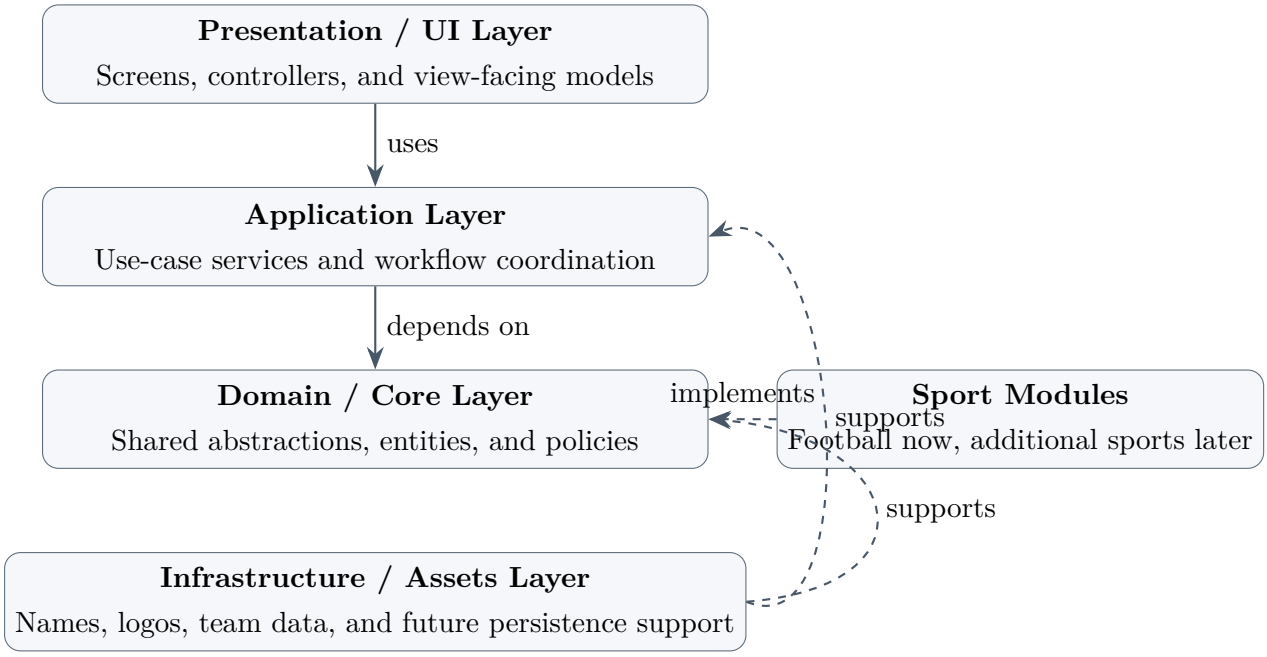

Figure 4.1: Layered architecture of PlayForge Manager

5 Core Abstractions and Interfaces

The core design is built around stable abstractions. These abstractions define what the system needs to do, while leaving sport-specific details to the Football module and future sport modules, which is a good thing.

5.1 Classification Overview

Table 5.1: Core abstractions and their roles

Abstraction	Role	Classification
`GameSession`	Holds the current playthrough state	Concrete class
`Sport`	Top-level playable sport contract	Interface
`SportFactory`	Creates and wires a sport-specific game world	Interface
`Ruleset`	Defines scoring and participation rules	Interface
`League`	Owns teams, fixtures, and competition context	Abstract class
`Season`	Coordinates progression across weeks	Abstract class
`Team`	Represents a managed club or squad	Abstract class
`Player`	Represents an athlete	Abstract class
`Coach`	Represents coaching staff	Abstract class
`Match`	Represents a played contest and its state	Abstract class
`Fixture`	Represents a scheduled pairing	Concrete class
`Lineup`	Represents the selected players for a match	Interface
`Tactic`	Represents strategic setup	Interface
`TrainingPlan`	Represents weekly training choices	Interface
`MatchEngine`	Simulates a match	Interface
`Scheduler`	Generates fixtures	Interface
`StandingsPolicy`	Computes league rankings	Interface
`InjuryPolicy`	Handles injuries and recovery	Interface
`AssetProvider`	Supplies names, logos, and team assets	Interface

5.2 Responsibility Summary

The abstractions follow a few clear rules.

- Entities own their own state. For example, `Team`, `Player`, and `Match` keep the data that belongs to them.
- Policies own algorithms that may vary by sport. This includes scheduling, standings, injuries, and match simulation.
- Factories own creation and wiring, not weekly game flow.
- The application layer owns workflow orchestration and management.

- The UI owns presentation only.

This means that `League` does not calculate rankings by itself, `Season` does not simulate matches by itself, and `MatchEngine` does not control season progression.

5.3 Representative Responsibility Matrix

The table under gives representative key attributes and methods together with the main dependents of each abstraction.

Table 5.2: Responsibilities, representative key attributes/methods, and main dependents

Type	Main responsibility	Representative key attributes / methods	Main dependents
`GameSession`	Holds current playthrough state	Active sport, current season, controlled team, progression state	Application layer, save/load support
`Sport`	Exposes the selected sport module	Access to ruleset, scheduler, standings, match engine, injury policy	Application layer, `SportFactory`
`SportFactory`	Creates the sport world	Build sport context, create session, season, and league	Game initialization
`Ruleset`	Defines sport rules	Scoring, squad limits, substitutions, points	Match engine, lineup validation
`Season`	Coordinates progression over time	Current week, advance operation	`GameSession`, application layer
`League`	Holds competition structure	Teams, fixtures, standings context	Season, standings service
`Team`	Holds a managed squad	Roster, selected tactic, selected lineup	Match preparation, squad view
`Player`	Holds athlete state	Identity, availability, injury state, attribute profile link	Team, match engine
`Coach`	Holds staff state	Role, specialization, training effect input	Team, training logic
`Match`	Holds played match state	Participants, lineups, tactics, result	Match engine, post-match logic
`Fixture`	Holds planned pairing	Week/round, home team, away team	League, season
`Lineup`	Represents selected players	Chosen starters and bench	Match, lineup service

Type	**Main responsibility**	**Representative key attributes / methods**	**Main dependents**
`Tactic`	Represents strategic setup	Tactical mode, style, formation-related values	Match, match engine
`TrainingPlan`	Represents weekly training choice	Focus, intensity, affected squad members	Team, injury logic
`MatchEngine`	Simulates a match	Run simulation using match context and rules	Match service
`Scheduler`	Generates fixtures	Create fixture calendar	League, season setup
`StandingsPolicy`	Computes rankings	Table update and tie-break handling	League, standings service
`InjuryPolicy`	Applies injury/recovery logic	Post-match and post-training injury handling	Match processing, weekly progression
`AssetProvider`	Supplies generated/loadable assets	Names, team names, logos	Initialization and UI support

5.4 Ownership Boundaries

To keep the architecture consistent, the most important abstractions also have explicit ownership limits.

Table 5.3: Ownership boundaries for key abstractions

Type	Owns or decides	Must not own or decide
`GameSession`	Active playthrough state, selected team, current season, current sport	Sport-specific rules, scheduling algorithm, standings algorithm
`SportFactory`	Creation and wiring of the selected sport world	Weekly progression, match results, standings updates during play
`Season`	Time progression at season level	Match simulation logic, standings rules
`League`	Teams, fixtures, competition structure	Standings calculation policy, match simulation logic
`Team`	Squad state, selected lineup/tactic references	Fixture generation, league ranking logic
`Match`	Match participants, lineups, tactics, result state	Multi-week progression, season ownership
`MatchEngine`	Match simulation procedure	Permanent match ownership, season state
`StandingsPolicy`	Ranking and tie-break rules	Team roster state, fixture generation
`InjuryPolicy`	Injury and recovery decisions	Lineup ownership, league standings

5.5 Key Design Notes

`GameSession` is the top-level state holder for one running playthrough. It keeps the selected sport, the user-controlled team, the active season, and the current progression state. This also makes it the natural anchor for any future save/load support.

`SportFactory` is the shared bootstrap abstraction. It creates the selected sport context, sets up the initial season environment, and uses `AssetProvider` during generation.

`Sport` exposes the policies and services a selected sport needs, such as `Ruleset`, `Scheduler`, `MatchEngine`, `StandingsPolicy`, and `InjuryPolicy`.

`Match` stores match-specific state, such as participants, lineups, tactical choices, and the final result. This keeps the match engine reusable and stateless.

`AssetProvider` is responsible for support data such as male and female names for players and coaches, team names, and logos.

6 Football Concrete Design

This section shows how Football fits into the shared abstractions from the previous section. The framework stays sport-neutral, while football-specific behavior is implemented through concrete classes.

6.1 Main Football Components

`FootballSportFactory`. This is the concrete implementation of the shared `SportFactory` abstraction used when Football is selected. The application layer depends only on `SportFactory`; football-specific initialization remains behind that interface.

`FootballSport`. This is the concrete implementation of `Sport`. It exposes football-specific services such as the ruleset, scheduler, standings policy, match engine, and injury policy. It does not coordinate weekly flow, initialize seasons, or manage top-level application logic.

`FootballRuleset`. This class implements `Ruleset` and defines football-specific rules such as scoring, match phases, lineup limits, point awards, and substitution rules.

`FootballSeason`. This is the football-specific specialization of `Season`. It keeps the football competition moving over weeks and works with the football league and fixtures created during initialization.

`FootballLeague`. This is the football-specific specialization of `League`. It stores the football teams and fixtures while delegating ranking logic to `FootballStandingsPolicy`.

`FootballScheduler`. This class implements `Scheduler` and produces the football fixture list. In the current design, a double round-robin league format is assumed.

`FootballStandingsPolicy`. This class implements `StandingsPolicy` and defines how football rankings are computed and how ties are resolved.

`FootballMatchEngine`. This class implements `MatchEngine`. It is designed as a *stateless* service. It receives a prepared match context, simulates the match according to the football ruleset, and returns outcomes to the match object and the application layer. It does not store permanent match state internally.

`FootballInjuryPolicy`. This class implements `InjuryPolicy` and applies football-

specific injury and recovery logic after training or matches.

6.2 Football Entities and Composition

The shared abstract entities are specialized by concrete football-side entities:

- `FootballMatch` extends `Match`
- `FootballTeam` extends `Team`
- `FootballPlayer` extends `Player`
- `FootballCoach` extends `Coach`
- `FootballLineup` implements `Lineup`
- `FootballTrainingPlan` implements `TrainingPlan`
- `FootballTactic` implements `Tactic`

`FootballPlayer` keeps football-specific data through composition rather than inheritance. In particular, it composes a `FootballAttributeProfile` and a football position model such as `FootballPosition`. This keeps football details out of the shared `Player` abstraction.

`FootballPosition`. This is a football-specific position model that distinguishes roles such as goalkeeper, defender, midfielder, and forward.

`FootballAttributeProfile`. This profile stores football-specific player attributes such as attack, defense, stamina, passing, and speed.

6.3 Football Initialization Notes

The top-level bootstrap abstraction is still the `SportFactory`. Football may use internal generation helpers during setup, but those helpers remain internal to the football module and are not treated as shared framework abstractions.

6.4 Mapping to the Core Model

Table 6.1: Football realization of shared abstractions

Core abstraction	Football realization
`SportFactory`	`FootballSportFactory`
`Sport`	`FootballSport`
`Ruleset`	`FootballRuleset`
`Season`	`FootballSeason`
`League`	`FootballLeague`
`Scheduler`	`FootballScheduler`
`StandingsPolicy`	`FootballStandingsPolicy`
`MatchEngine`	`FootballMatchEngine`
`InjuryPolicy`	`FootballInjuryPolicy`
`Lineup`	`FootballLineup`
`TrainingPlan`	`FootballTrainingPlan`
`Match`	`FootballMatch`
`Team`	`FootballTeam`
`Player`	`FootballPlayer`
`Coach`	`FootballCoach`
`Tactic`	`FootballTactic`

7 Future Sport Extension Strategy

The system is not meant to stay football-only. A second sport will be introduced later in the semester, so the architecture must already leave room for it. In this document, the future sport example is Volleyball.

To add Volleyball, the system would introduce a new sport module with classes such as `VolleyballSportFactory`, `VolleyballSport`, `VolleyballRuleset`, `VolleyballSeason`, `VolleyballLeague`, `VolleyballScheduler`, `VolleyballStandingsPolicy`, `VolleyballMatchEngine`, `VolleyballInjuryPolicy`, `VolleyballLineup`, and `VolleyballTrainingPlan`. It would also provide concrete entities such as `VolleyballTeam`, `VolleyballPlayer`, `VolleyballCoach`, and `VolleyballMatch`.

The following parts of the system should stay the same when a new sport is added:

- the overall UI structure,

- the application services,
- the shared domain abstractions,
- the top-level game/session model,
- and the fixture concept as a scheduled pairing.

At the same time, the framework should not assume that every sport behaves like football during a match. A future sport may introduce match phases, break-based decision points, or in-match substitutions that happen differently. The design therefore treats the top-level workflow as generic and leaves sport-specific internal match flow inside the selected sport module, mainly behind `Ruleset`, `MatchEngine`, and any sport-specific phase or substitution logic.

8 UI Architecture and Screen Map

The UI is considered as the presentation layer. It uses application services and view-facing data, not concrete football classes. This means the interface can keep the same general shape even when another sport is added later.

The planned desktop UI technology is JavaFX as talked about before. Still, the screen descriptions below are written in framework-neutral terms so that the design stays focused on responsibilities rather than widget details.

Table 8.1: Screen map and service usage

Screen	Purpose	Application service	View-facing data
Home / New Game	Starts a new session	Game Initialization Service	Start options
Sport Selection	Lets the user choose a sport	Sport Selection Service	Sport summaries
Team Overview	Shows team status and summary	Team Overview Service	Team overview data
Squad / Player List	Shows squad members and conditions	Squad Service	Player summaries
Tactics / Lineup	Lets the user choose lineup and tactic	Lineup Service	Lineup and tactic data
Fixture List	Shows upcoming and completed matches	Fixture Service	Fixture summaries
League Table	Shows standings	Standings Service	Standings rows
Match Screen	Shows preparation and match state	Match Service	Match state data
Post-Match Summary	Shows result and key updates	Post-Match Service	Match result summary

The screens are intentionally generic. A future sport may change the content shown on them, but it should not require a different screen architecture.

9 Main Workflows

This section shows the main user and system flow in three activity-style workflows and one sequence-style interaction view.

9.1 New Game Initialization

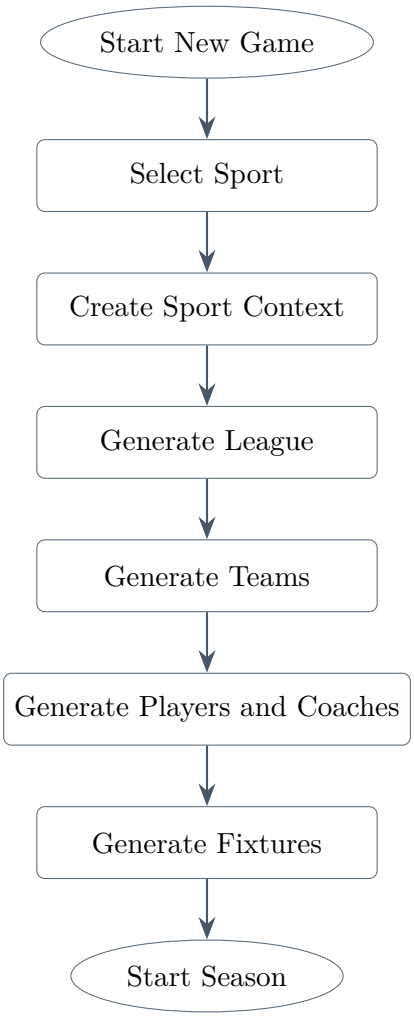

Figure 9.1: Workflow 1: New game initialization

9.2 Weekly Progression

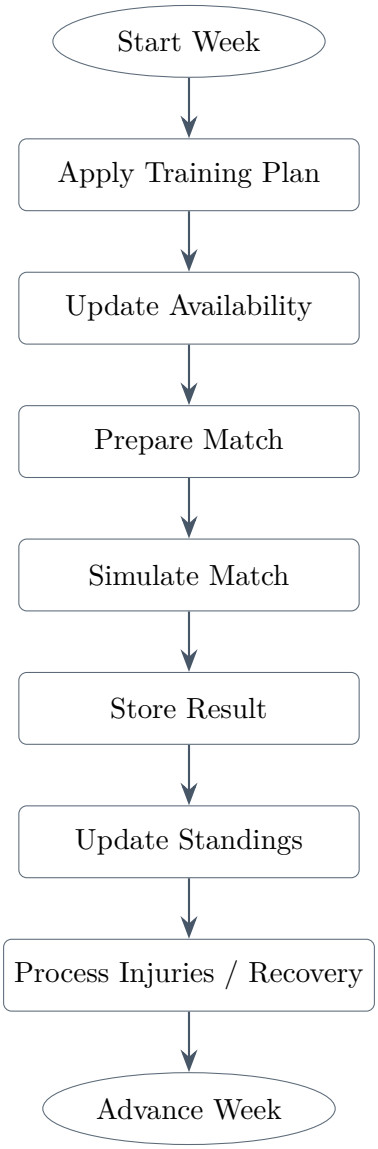

Figure 9.2: Workflow 2: Weekly progression

9.3 Match Preparation, Simulation, and Result Update

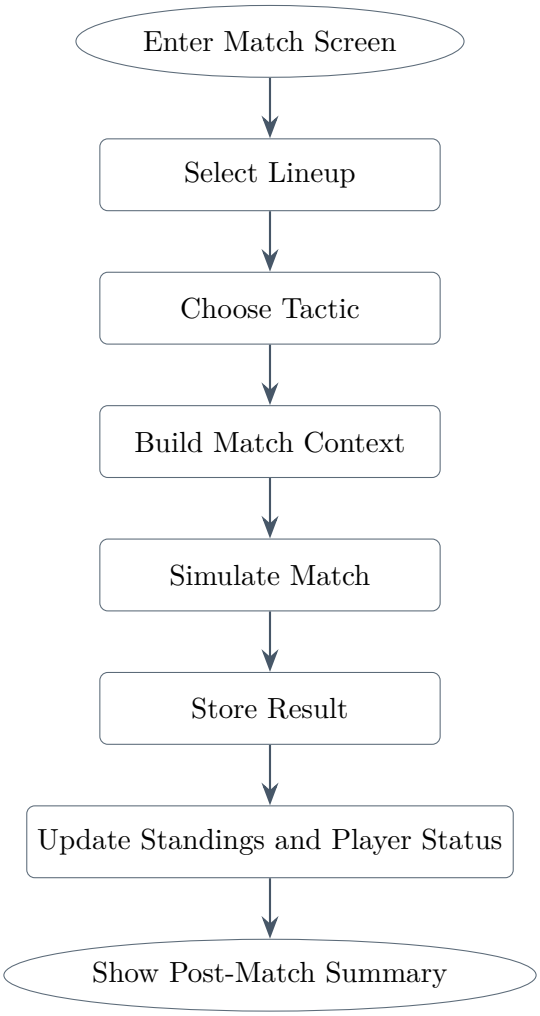

Figure 9.3: Workflow 3: Match preparation, simulation, and result update

9.4 Sequence View of New Game Initialization

The next figure shows one important interaction chain in more detail. Its role is to make the UI/application/domain boundary visible. In particular, it shows that the UI talks to an application service, the application service talks to the shared factory abstraction, and the sport-specific behavior stays behind that abstraction.

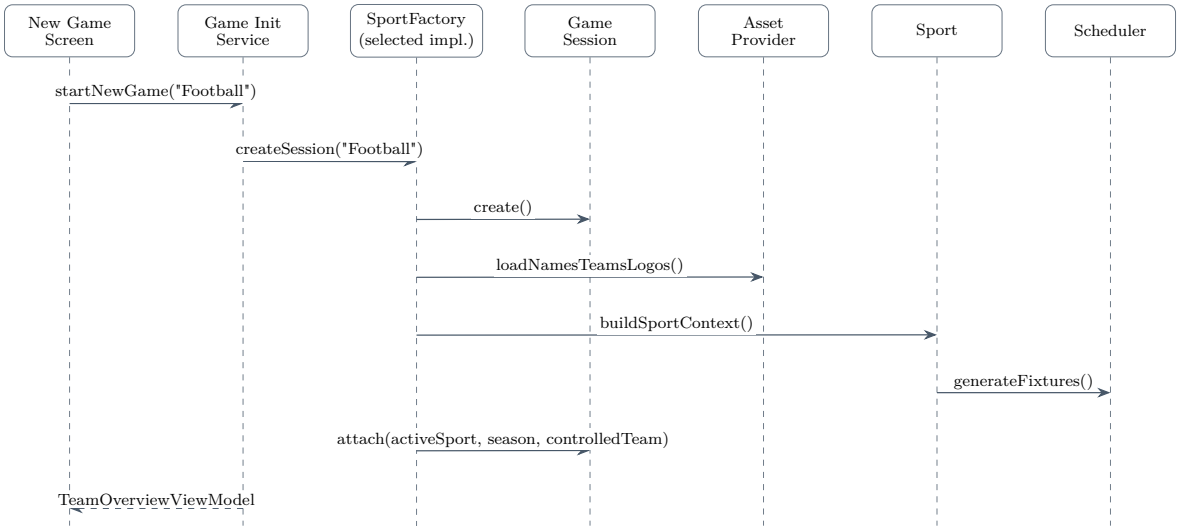

Figure 9.4: Sequence view: New game initialization through the application layer

This figure does not try to show every internal message. Its main job is to show the architectural boundary clearly: the UI does not call Football classes directly, and the application layer uses abstractions to reach the selected sport module.

10 Class Relationships

This section gives the main UML view of the framework. The model is shown in two linked figures for readability. Figure 10.1 shows the shared framework structure, including the top-level `GameSession`. Figure 10.2 shows the football-side realizations of the shared abstractions, including the football-specific season, league, factory, lineup, and training plan classes. Together, these two figures form the main class diagram of the design.

10.1 Shared Framework Relationships

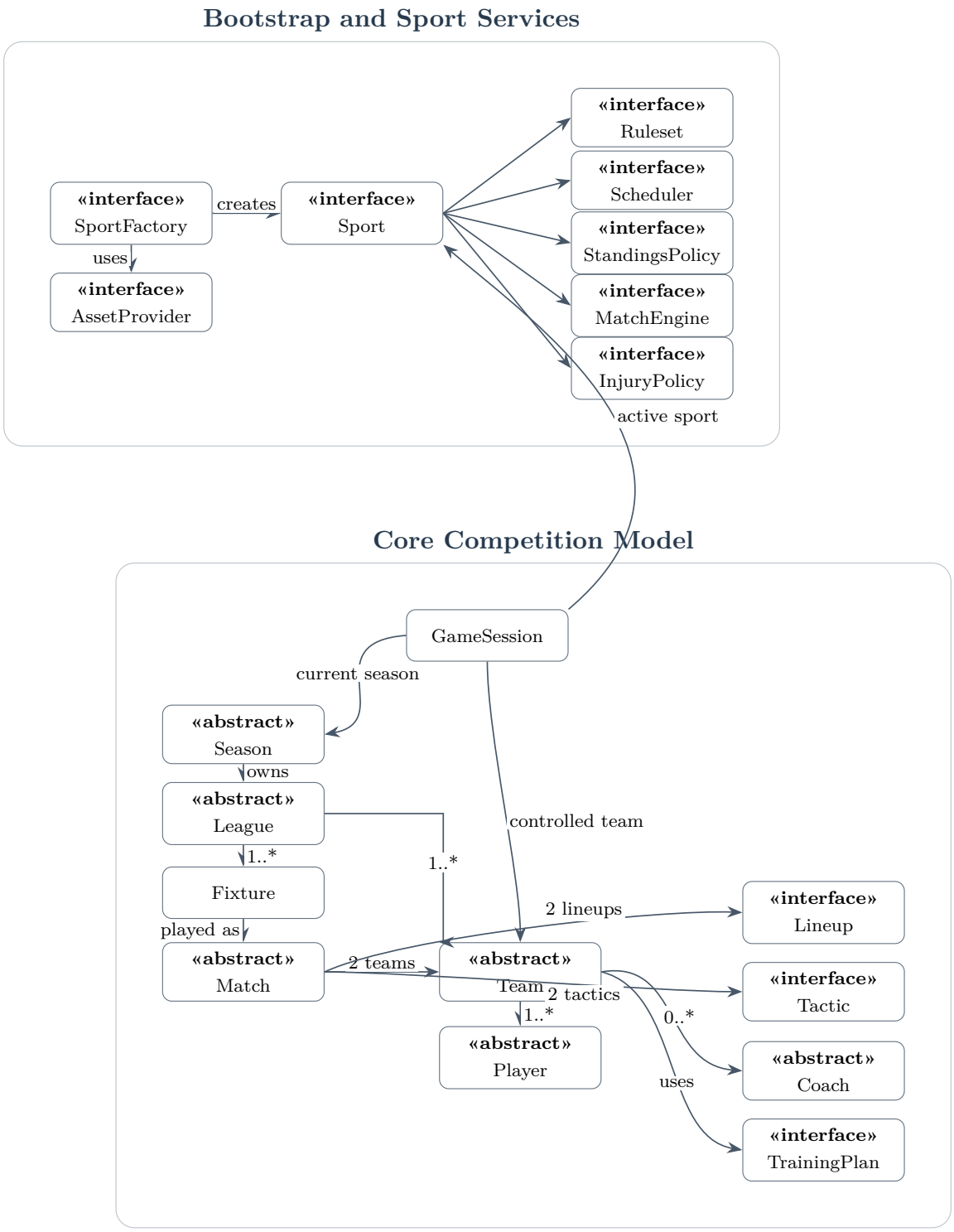

Figure 10.1: Shared framework class relationships

10.2 Football Realization of the Framework

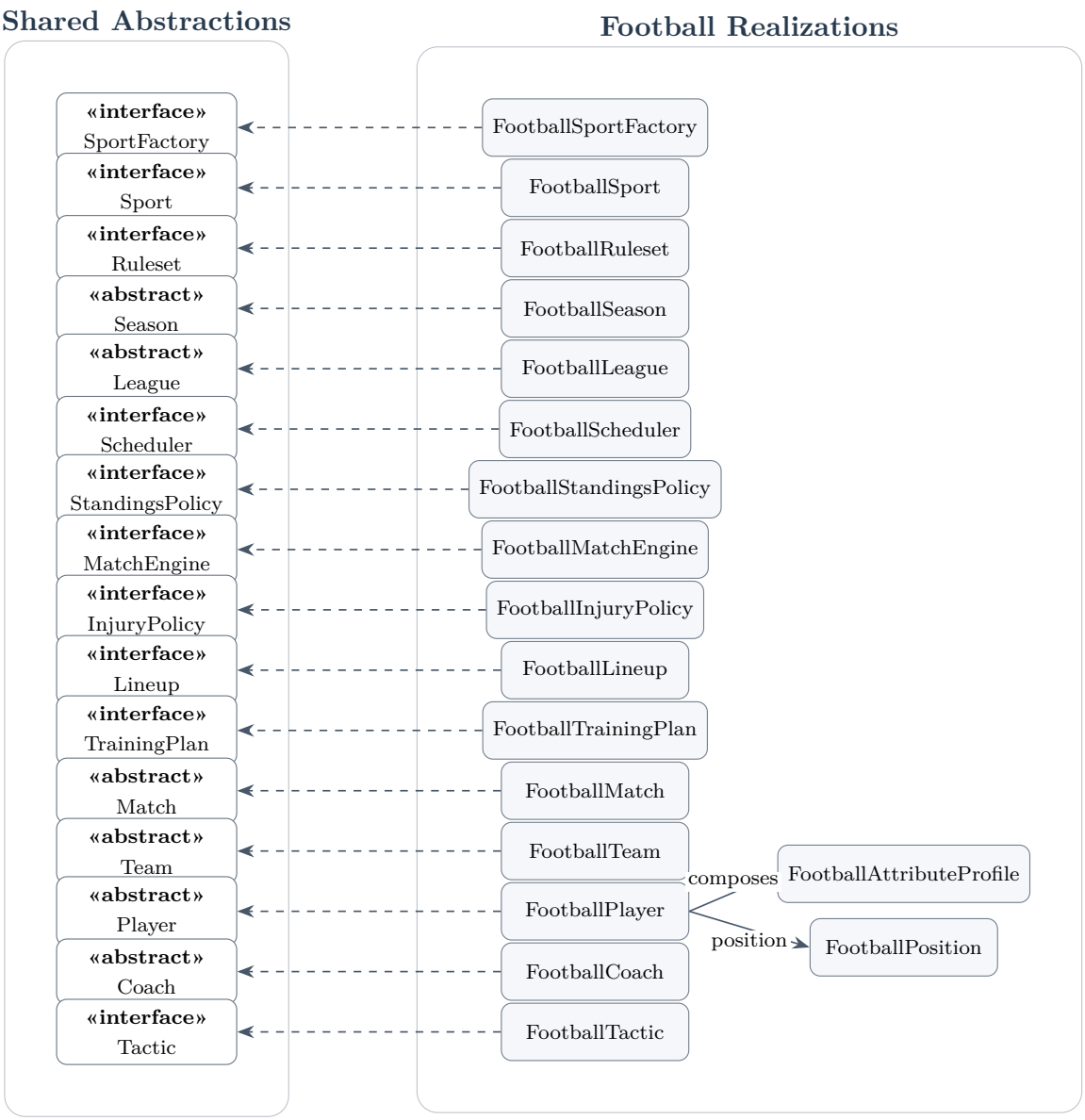

Figure 10.2: Football realization of the shared framework

Together, Figures 10.1 and 10.2 provide the main UML view of the system in split form. This makes the model easier to read while still showing inheritance, interface realization, composition, and the main associations.

11 OO Principles and Design Decisions

The design choices in this design document follow standard object-oriented and software architecture practice [1, 2, 3]. The point is not to apply design principles in a mechanical

way, but to use them where they genuinely help the design.

Abstraction. The system works with shared concepts such as `Sport`, `GameSession`, `Team`, `Match`, and `Scheduler` rather than football-only classes.

Encapsulation. Each class owns a clear piece of state and exposes behavior through controlled operations instead of exposing internals directly.

Open/Closed Principle. The framework is open for extension but closed for invasive redesign. A new sport is added by implementing existing abstractions such as `SportFactory`, `Sport`, `Ruleset`, `Scheduler`, and `MatchEngine` rather than rewriting the whole system.

Dependency Inversion. High-level parts of the system, especially the UI and application layer, depend on abstractions rather than concrete football classes. This is shown both in the layered architecture and in the sequence view.

Single Responsibility Principle. The design keeps responsibilities separate. For example, `StandingsPolicy` computes rankings, `Scheduler` generates fixtures, `MatchEngine` simulates matches, and `SportFactory` handles initialization.

Composition Over Inheritance. Inheritance is used for stable specialization such as `FootballPlayer` extending `Player`. Composition is used where behavior or data can vary, such as `FootballPlayer` composing `FootballAttributeProfile`.

12 Risks, Assumptions, and Open Questions

12.1 Risks

Table 12.1: Main design risks

ID	Risk	Impact	Mitigation
R1	Concrete football classes may leak into the UI or application layer	High	Enforce dependency rules and review imports carefully
R2	Match simulation may be harder to generalize across sports than expected	High	Keep `MatchEngine` broad and keep details inside sport modules
R3	The second sport may be announced late in the semester	High	Build football through abstractions from the beginning
R4	Different sports may require UI presentation changes	Medium	Keep screens generic and limit sport-specific formatting to display data
R5	The abstraction layer may become more complex than needed	Medium	Keep interfaces narrow and remove unnecessary detail during implementation

12.2 Assumptions

The current design assumes one main league per season. It also assumes a manager-style game without a full financial model such as transfers, salaries, or budgets.

The football design assumes a double round-robin season format. The project also assumes that names, team names, and logos are supplied through a support abstraction rather than being handled directly inside the UI or core domain classes.

13 Rubric and Requirement Traceability

This section maps the milestone expectations to the relevant parts of the document so that coverage is clear and easy to review.

Table 13.1: Milestone traceability

Criterion / requirement	Sections	How it is addressed
Comprehensive design document	1–14	The report covers scope, architecture, abstractions, football design, UI, workflows, relationships, principles, risks, traceability, and process coverage
Abstraction-based design	3, 4, 10	The architecture and the class model are built around interfaces and abstract classes
Football included as the first sport	2, 6, 9, 10	Football is the first concrete module and is used throughout the workflow and class model
Ready for another sport later	3, 5, 7, 11	The design keeps sport-specific logic behind abstractions and provides a clear extension path
UI separated from sport-specific logic	4, 8, 9	The UI talks to application services and view-facing data rather than concrete football classes, and the sequence view makes that boundary visible
Classes, components, and relationships shown clearly	4, 9, 10	The report includes a layered architecture figure, workflow figures, a sequence diagram, and UML-style class diagrams

14 Design Process and Ownership

14.1 Initial Design Baseline

At the first milestone meeting, the team decided on the basic architectural direction before detailed writing began. The report would focus on design and architecture only. The system would use five layers, Presentation/UI, Application, Domain/Core, Sport Modules, and Infrastructure/Assets. The UI would depend on application services and domain-facing abstractions rather than concrete football classes. Football would be the first concrete module, Volleyball would be the placeholder future sport example, and the workflow set would be limited to a small number of diagrams.

The agreed task plan also divided the work into four streams that are reflected in the final document: document structure and architecture direction, core abstractions and UML, football/UI/workflow design, and extension strategy/risk/rubric review. In that sense, the final report keeps the same work split as the original milestone plan, but presents it

as single merged document.

14.2 Section and Diagram Ownership

Table 14.1: Section and diagram ownership

Owner	Main sections	Diagram responsibility
Mirza	Scope, architecture direction, integration	Architecture diagram and final merge
Sepehr	Core abstractions and UML/class model	Main UML/class-relationship diagrams
Doruk	Football design, UI structure, workflows	Workflow diagrams
Çağan	Extension strategy, principles, risks, traceability	Quality review for extensibility and rubric fit

Conclusion

This design gives PlayForge Manager quite a solid structure for the next milestones. The system is organized around shared abstractions, clear layer boundaries, and a Football module that shows how the framework is supposed to grow. Match state stays in the match model, the match engine stays stateless, the UI stays separate from football-specific classes, and the system leaves room for another sport later without having to be rebuilt.

Declaration on the Use of AI Assistance

During the preparation of this report, we used AI-based tools in a limited support role. These tools were used for brainstorming, wording refinement, consistency checks, and help with LaTeX and diagram code. The final design decisions, the technical content, and the submitted version of the report remain the responsibility of the team.

References

[1] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.

[2] Robert C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2002.

[3] Ian Sommerville, *Software Engineering*, 10th ed., Pearson, 2015.